



Objective & Lecture Mapping: In previous labs, we built a Simple Reflex Agent that moved randomly or reacted only to immediate adjacent cells. Today, we transition to a **Goal-Based/Planning Agent**. You will expose the environment's state space to the agent, allowing it to use **Breadth-First Search (BFS)**, **Depth-First Search (DFS)**, and **Uniform-Cost Search (UCS)** to simulate and find paths to the food before taking a physical action.

Part 1: Practical Implementation — Building the Search Agent

Step 1.1: Exposing the World Model

Classical search algorithms require an abstract model of the environment to simulate future states. Currently, your agent is "blind" to the overall map.

1. Create a new Branch called Week_03 in your Forked Github Repo and work on the below Questions.
2. Open `visual_grid_game.py` .
3. Locate the `get_percept(self)` method.
4. Add three new keys to the dictionary to pass the global state to the agent:

```
'grid_size': (self.width, self.height)
'walls': list(self.walls)
'all_food': list(self.food_positions)
```

Step 1.2: Implementing BFS, DFS, and UCS

You will implement three distinct uninformed search strategies. The core node expansion structure is identical; only the data structure for the **Frontier** changes!

1. Open `agent.py` and create a new class called `SearchAgent` .
2. Import required structures: `from collections import deque` and `import heapq` .
3. **BFS:** Create `def bfs_search(...)` . Use a FIFO queue (`deque.popleft()`). This explores the shallowest nodes first.
4. **DFS:** Create `def dfs_search(...)` . Use a LIFO stack (`list.pop()`). This explores the deepest nodes first.
5. **UCS:** Create `def ucs_search(...)` . Use a Priority Queue (`heapq.heappop()`) ordered by the total path cost $g(n)$.
6. For all three algorithms, you must maintain a `reached` set (or visited array) to track explored states. This converts your algorithm from a *Tree Search* to a *Graph Search*, preventing infinite loops.

Step 1.3: Executing and Comparing Offline Plans

The agent must now form a complete plan and execute it step-by-step.

1. In your `SearchAgent.__init__` , add an empty list: `self.plan = []` and a config string `self.active_algo = 'BFS'` .
2. In `sense_and_act(self, percept)` , check if `self.plan` is empty.
3. If empty, find the closest food pellet from `percept['all_food']` , execute the search method matching `self.active_algo` , and store the resulting sequence of actions in `self.plan` .
4. Return the first action from the plan using `return self.plan.pop(0)` .

5. **Observation Task:** Change `self.active_algo` between 'BFS', 'DFS', and 'UCS' and run the simulation. Watch how DFS takes erratic, winding paths, while BFS and UCS take direct, optimal paths!

Part 2: Theoretical Evaluation

Based on your implementation and Lecture 04, complete the following analytical questions.

1. (Remember) You built your search logic based on the environment coordinates. What is the fundamental difference between the "State Space" and the "Search Tree"?

The **State Space** is the complete set of all possible states and transitions that can exist in the environment. In our grid, each valid coordinate can represent a state. The **Search Tree** is the structure generated by the search algorithm as it explores these states from the initial state toward the goal. Therefore, the state space describes what states are possible, while the search tree describes how the algorithm explores those states.

2. (Understand) In Step 1.2, you were instructed to use a `reached` set. In graph search theory, what specific problem does this set solve, and what would happen to your DFS agent without it?

The `reached` set prevents the search algorithm from repeatedly exploring the same states. It keeps track of states that have already been reached, avoiding unnecessary repeated exploration and infinite loops. Without the `reached` set, the DFS agent could keep moving between already visited grid cells and enter an infinite loop, especially when the search encounters cycles in the grid.

3. (Analyze) Compare the visual paths generated by BFS and DFS in your simulation. Why is BFS guaranteed to be optimal in this specific grid, whereas DFS produces winding, suboptimal routes?

In our simulation, **BFS produces a direct and shortest path** to the food because it explores the grid level by level, starting from the current position. Since every movement in our grid has the same cost of 1, the first time BFS reaches the food, it has found the path with the minimum number of moves. Therefore, BFS is guaranteed to be optimal in this specific grid.

DFS, on the other hand, explores one branch as deeply as possible before backtracking. It does not consider whether another path is shorter, so it can explore unnecessary cells and produce winding or suboptimal routes. Therefore, DFS can reach the food but is not guaranteed to find the shortest path.

4. (Evaluate) If we scaled `visual\_grid\_game.py` up to a massive 1000x1000 grid, BFS and UCS might crash before finding food. According to the time and space complexity formulas from the lecture, contrast the memory bottlenecks of BFS versus DFS.

BFS has a much larger memory requirement than DFS because BFS stores all the nodes at the current search depth in its frontier. Its space complexity is approximately $O(b^d)$, where b is the branching factor and d is the depth of the shallowest solution. As the grid becomes larger, the number of nodes stored in the queue can grow rapidly, making memory the main bottleneck for BFS.

DFS uses much less memory because it mainly stores the current path and the unexplored branches. Its space complexity is approximately $O(bm)$, where b is the branching factor and m is the maximum search depth. Therefore, DFS can handle larger search spaces with less memory, although it may take a much longer or suboptimal route to find the food.

Therefore, for a massive grid, **BFS is more likely to run out of memory because of its large frontier**, while **DFS is more memory-efficient but sacrifices path optimality**.

Once Completed Get a PDF of the completed File and Push into the Week 03 brach in the Github Project

